

NYCU Introduction to Machine Learning, Homework 3

[112550069], [謝佑哲]

Part. 1, Coding (60%): (20%) Adaboost

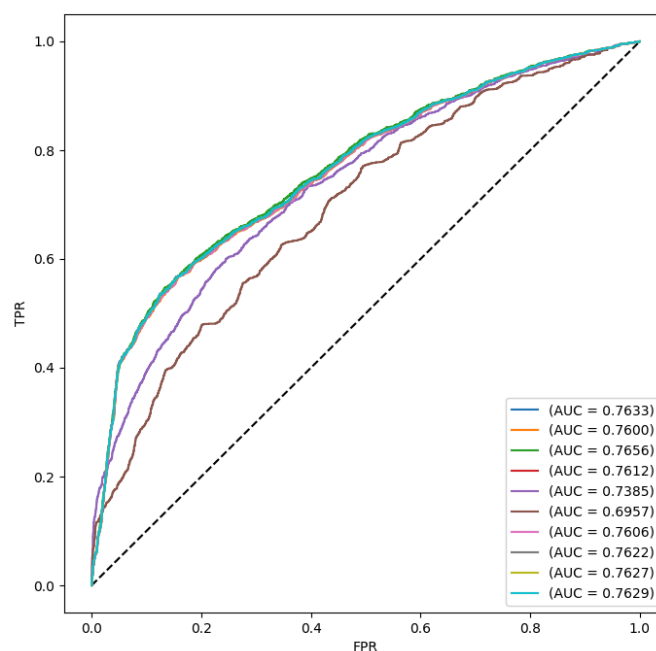
1. (10%) Show your accuracy of the testing data ($n_estimators = 10$)

```
# AdaBoost
clf_adaboost = AdaBoostClassifier(
    input_dim=X_train.shape[1],
)
```

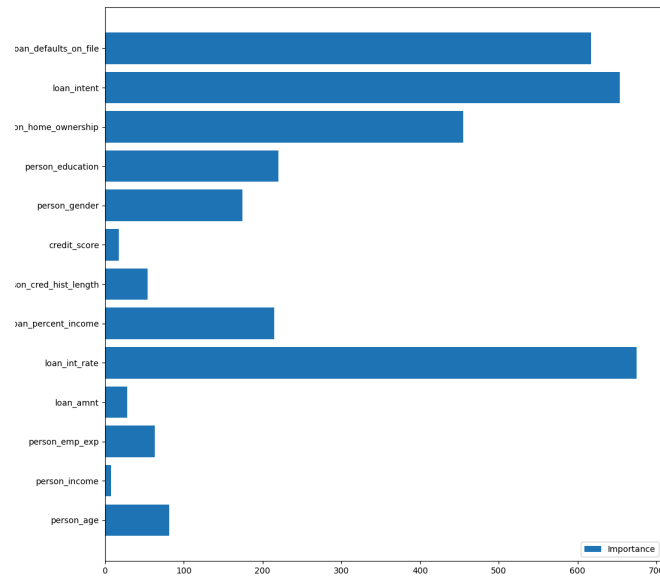
```
class AdaBoostClassifier:
    def __init__(self, input_dim: int, num_learners: int = 10) -> None:
        self.sample_weights = None
        # create 10 learners, dont change.
        self.learners = [
            WeakClassifier(input_dim=input_dim) for _ in range(num_learners)
        ]
```

2024-11-17 17:59:36.437 | INFO | __main__:main:54 - AdaBoost - Accuracy: 0.8239

2. (5%) Plot the AUC curves of each weak classifier.



3. (5%) Plot the feature importance of the AdaBoost method. Also, you should snapshot the implementation to calculate the feature importance.



Basically, I calculated feature importance by absolute value of weight. Because I use one-hot encoded on class features, I calculate the feature importance by means of sub columns of the class.

```
def compute_feature_importance(self, df) -> t.Sequence[float]:
    """Implement your code here"""
    alpha_sum = torch.sum(torch.stack(self.alphas))
    idx = 0
    feature_importance = torch.zeros(self.learners[0].model[0].weight.shape[1]).to(device)
    for model in self.learners:
        feature_importance += torch.abs(model.model[0].weight.T @ model.model[1].weight.T \
            * self.alphas[idx] / alpha_sum).view(-1) # (27,mid_dim) @ (mid_dim,1) = (27,1)
        idx += 1

    feature_name = ['person_age', 'person_income', 'person_emp_exp', 'loan_amnt', 'loan_int_rate',
        'loan_percent_income', 'cb_person_cred_hist_length', 'credit_score', 'person_gender',
        'person_education', 'person_home_ownership', 'loan_intent', 'previous_loan_defaults_on_file']
    # integrate the one hot encoded result to original result
    person_gender = torch.mean(feature_importance[8:10]).view(-1)
    person_education = torch.mean(feature_importance[10:15]).view(-1)
    person_home_ownership = torch.mean(feature_importance[15:19]).view(-1)
    loan_intent = torch.mean(feature_importance[19:25]).view(-1)
    previous_loan_defaults_on_file = torch.mean(feature_importance[25:]).view(-1)
    feature_importance = torch.cat((feature_importance[:8], person_gender, person_education,
        person_home_ownership, loan_intent, previous_loan_defaults_on_file))
    return feature_importance, feature_name
```

(20%) Bagging

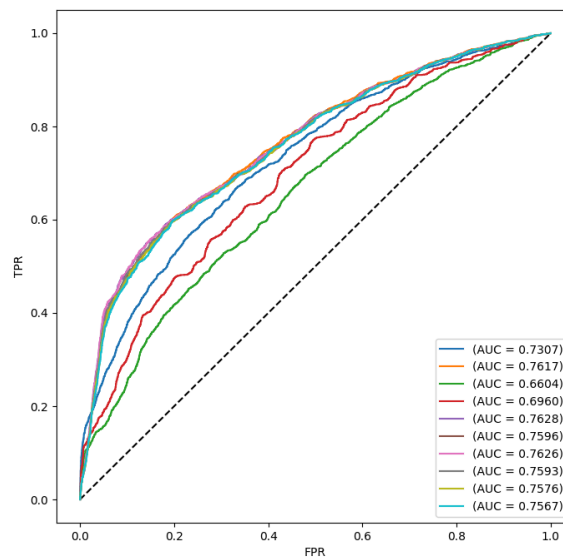
- (10%) Show your accuracy of the testing data with 10 estimators. (n_estimators=10)

```
# Bagging
clf_bagging = BaggingClassifier(
    input_dim=X_train.shape[1],
)
```

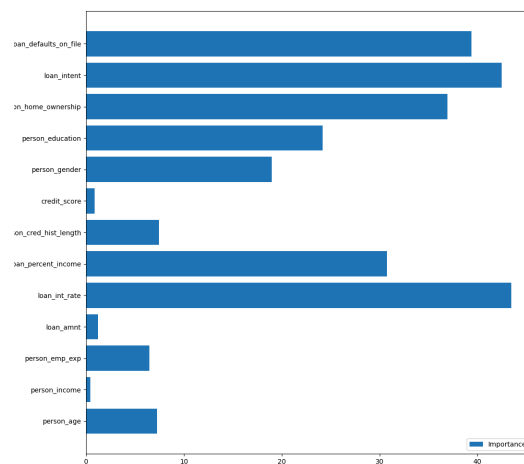
```
class BaggingClassifier:
    def __init__(self, input_dim: int) -> None:
        # create 10 learners, dont change.
        self.learners = [
            WeakClassifier(input_dim=input_dim) for _ in range(10)
        ]
```

2024-11-17 18:00:13.396 | INFO | __main__:main:78 - Bagging - Accuracy: 0.8017

- (5%) Plot the AUC curves of each weak classifier.



- (5%) Plot the feature importance of the Bagging method. Also, you should snapshot the implementation to calculate the feature importance.



Basically, I calculated feature importance by absolute value of weight.

Because I use one-hot encoded on class features, I calculate the feature importance by means of sub columns of the class.

```
def compute_feature_importance(self) -> t.Sequence[float]:
    """Implement your code here"""
    feature_importance = torch.zeros(self.learners[0].model[0].weight.shape[1]).to(device)
    for model in self.learners:
        feature_importance += torch.abs(model.model[0].weight.T @ model.model[1].weight.T).view(-1) # (2
    feature_name = ['person_age', 'person_income', 'person_emp_exp', 'loan_amnt', 'loan_int_rate',
                    'loan_percent_income', 'cb_person_cred_hist_length', 'credit_score', 'person_gender',
                    'person_education', 'person_home_ownership', 'loan_intent', 'previous_loan_defaults_on_file']
    # integrate the one hot encoded result to original result
    person_gender = torch.mean(feature_importance[8:10]).view(-1)
    person_education = torch.mean(feature_importance[10:15]).view(-1)
    person_home_ownership = torch.mean(feature_importance[15:19]).view(-1)
    loan_intent = torch.mean(feature_importance[19:25]).view(-1)
    previous_loan_defaults_on_file = torch.mean(feature_importance[25:]).view(-1)
    feature_importance = torch.cat((feature_importance[:8], person_gender, person_education,
                                    person_home_ownership, loan_intent, previous_loan_defaults_on_file))
    return feature_importance, feature_name
```

(15%) Decision Tree

7. (5%) Compute the Gini index and the entropy of the array [0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1].

```
2024-11-17 18:00:13.766 | INFO | __main__:main:93 - Sample gini: 0.4628
```

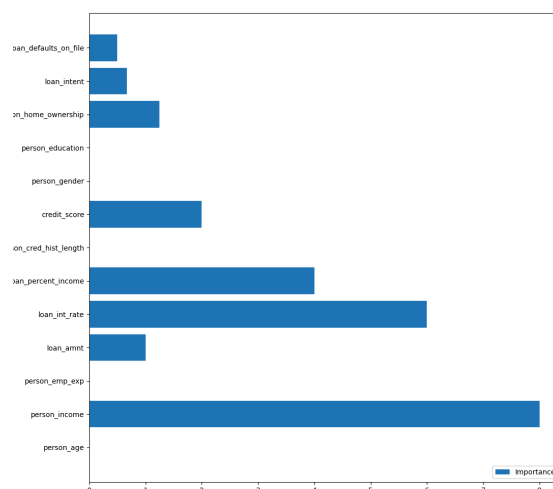
```
2024-11-19 20:27:13.016 | INFO | __main__:main:92 - Sample entropy: 0.6555
```

8. (5%) Show your accuracy of the testing data with a max-depth = 7

```
# Decision Tree
clf_tree = DecisionTree(
    max_depth=7,
)
```

```
2024-11-17 18:00:56.924 | INFO | __main__:main:102 - DecisionTree - Accuracy: 0.9124
```

9. (5%) Plot the feature importance of the decision tree.



```
def get_feature_importance(self):
    feature_importance = self.feature_importance
    feature_name = ['person_age', 'person_income', 'person_emp_exp', 'loan_amnt', 'loan_int_rate',
                    'loan_percent_income', 'cb_person_cred_hist_length', 'credit_score', 'person_gender',
                    'person_education', 'person_home_ownership', 'loan_intent', 'previous_loan_defaults_on_file']

    # integrate the one hot encoded result to original result
    person_gender = torch.mean(feature_importance[8:10]).view(-1)
    person_education = torch.mean(feature_importance[10:15]).view(-1)
    person_home_ownership = torch.mean(feature_importance[15:19]).view(-1)
    loan_intent = torch.mean(feature_importance[19:25]).view(-1)
    previous_loan_defaults_on_file = torch.mean(feature_importance[25:]).view(-1)
    feature_importance = torch.cat((feature_importance[:8], person_gender, person_education,
                                    person_home_ownership, loan_intent, previous_loan_defaults_on_file))

    return feature_importance, feature_name
```

(5%) Code Linting

10. Show the snapshot of the flake8 linting result (paste the execution command even when there is no error).

```
• (DLP) youzhe0305@gpu7:~/NYCU-Intro-ML/hw3$ flake8 main.py
• (DLP) youzhe0305@gpu7:~/NYCU-Intro-ML/hw3$ flake8 src/adaboost.py
• (DLP) youzhe0305@gpu7:~/NYCU-Intro-ML/hw3$ flake8 src/decision_tree.py
• (DLP) youzhe0305@gpu7:~/NYCU-Intro-ML/hw3$ flake8 src/bagging.py
• (DLP) youzhe0305@gpu7:~/NYCU-Intro-ML/hw3$ flake8 src/utils.py
```

Part. 2, Questions (40%):

1. (10%) What are Bagging and Boosting, and how are they different? Please list their differences and compare the two methods in detail.

Both Bagging and Boosting methods try to make data points have different weights and train a lot of weak learners by distinct weighted distribution. However, different algorithms are designed for them to achieve the purpose.

1. Bagging generates a lot of dataset by randomly sampling with replacement. Each dataset may have some element appear for many times, which can be seen that those data points are given more weight. Data weights of boosting methods are determined by previous trained models. When previous trained models misclassify a data point, the weight of this data point will increase adaptively. The main difference is that the data weight is determined randomly or adaptively by previous result.

2. Each weak learner will contribute the same extent to the final result (eg: have equivalent vote right). Nevertheless, boosting methods' weak learners have different weight to contribute to the final result by the loss of weak learners. If losses of weak learners are lower, the weight they get for contribution will be bigger.

3. Based on previous properties, bagging is more suitable for those weak learners which are easy to overfit because it will not try to decrease loss as much as possible. It just randomly increases models' diversity to increase robustness. On the other hand, boosting is more suitable for weak learners whose capacities are not enough and are not easy to overfit because it pursues low loss as much as possible.

2. (15%) What criterion do we use to decide when we stop splitting while performing the decision tree algorithm? Please list at least three criteria.

There are 4 criteria to decide whether stop splitting nodes of decision tree:

1. whether the data subset of a node is pure, which means all data point are same class
2. whether the node's depth reaches the maximum depth.
3. whether the node's data subset is smaller than the minimum number of data.
4. whether information gain (entropy reduced) is less than a threshold.

3. (15%) A student claims to have a brilliant idea to make random forests more powerful: since random forests prefer trees which are diverse, i.e., not strongly correlated, the student proposes setting $m = 1$, where m is the number of random features used in each node of each decision tree. The student claims that this will improve accuracy while reducing variance. Do you agree with the student's claims? Clearly explain your answer.

No, I don't agree with the students' claims. Although set $m = 1$ can increase the diversity of random forests, it will also strongly restrict node's feature choice. The only feature that can be chosen may be not representative and this situation will weaken the ability of a decision tree. Despite diversity increasing, the much weaker decision trees will make the whole random forests weaker. Thus, we should set a more proper m rather than $m = 1$ such an extreme number.